PROJECT REPORT

LAKSHYA

MULTIMODAL AI CHATBOT

An Intelligent Personal AI Assistant for Text, Retrieval, Vision, and Voice Interaction

Submitted by	Akanksha Gupta
Institution	Shri Ramswaroop Memorial University, Lucknow
Project Guide	Faculty Advisor / Project Evaluation Committee
Academic Year	2026–2027
Repository	github.com/akankshagupta20052007-del/Lakshya_Multimodal_AI_CHATBOT

Submitted in partial fulfillment of the academic project requirements.

ABSTRACT

LAKSHYA is a production-ready multimodal conversational AI assistant designed to provide a unified conversational interface for interacting with large language models, retrieval-based knowledge systems, uploaded files, and media-related tools. The project is implemented in Python, leveraging a high-performance FastAPI backend API and an interactive executive-themed Streamlit user interface. Google Gemini (gemini-3.5-flash-lite) powers the language model reasoning, visual scene comprehension, and speech transcription pipelines. Retrieval-Augmented Generation (RAG) is supported through document chunking and vector/keyword indexing, ensuring verifiable citations. Dedicated intent-based routing dispatches user requests among conversational chat, document analysis, prompt-adherent image generation via Pollinations AI with Gemini prompt intelligence, and voice synthesis via Edge-TTS and gTTS without returning unrelated stock photo fallbacks. The system has been validated across automated unit tests and synchronized to a central GitHub version control repository.

Keywords: *Multimodal AI, Chatbot, FastAPI, Streamlit, Gemini, RAG, Intent Routing, Speech Processing, Neural Image Generation.*

1. INTRODUCTION

Conversational artificial intelligence has rapidly evolved from text-constrained dialog systems into comprehensive assistants capable of processing multiple modalities. However, typical conversational applications often isolate functions: users are forced to juggle separate software for asking questions, retrieving private documents, understanding visual images, recording audio memos, or generating artwork. A multimodal assistant brings these capabilities together into a single, cohesive workflow.

LAKSHYA (Multimodal AI Chatbot) is engineered around a modular architecture that cleanly decouples the presentation layer, the REST API backend, the language model inference engine, the knowledge retrieval mechanism, and specialized media services. This clear separation of concerns ensures that each service operates reliably, scales independently, and handles service-level failures gracefully.

2. PROBLEM STATEMENT

Modern users encounter friction when navigating between disparate applications for basic digital tasks: querying knowledge bases, extracting insights from uploaded PDFs, generating relevant images from descriptions, and dictating voice prompts. Furthermore, naive conversational systems often suffer from intent confusion—such as misinterpreting an explicit image generation request as a web search, or returning static, unrelated stock photos when an external provider experiences cooldown delays. LAKSHYA solves these challenges through explicit intent routing, unified input staging, Gemini prompt enrichment, and strict prompt-adherent generation pipelines.

3. OBJECTIVES

- Provide a unified, single-chatbar conversational interface with a direct inline microphone recorder and modal tools dock.
- Integrate Gemini LLM for high-accuracy reasoning, contextual chat history, and vision-language comprehension.
- Implement a Retrieval-Augmented Generation (RAG) pipeline supporting PDF, TXT, and MD document ingestion with verifiable citations.
- Enable native microphone voice capture with automatic Gemini audio transcription and neural speech playback.
- Build a strict prompt-adherent image generation service with Gemini prompt intelligence that eliminates typos and stock photo fallbacks.
- Ensure resilient dual-mode database storage (MySQL with automatic fallback to SQLite) to prevent connection failures.
- Enforce safety controls, input sanitation, and crisis hotline detection (988 Lifeline).
- Maintain automated test suites and deploy full version control to GitHub.

4. PROJECT SCOPE

The initial scope encompasses a fully functional local studio combining a Streamlit frontend and FastAPI backend. The architecture is designed for turnkey transition to cloud containers and Kubernetes clusters.

In-Scope Capabilities: Natural language conversation; contextual history; RAG document grounding; JPEG/PNG vision analysis; neural text-to-image synthesis with prompt enrichment; speech transcription; text-to-speech audio output; emotional support dialogues; and automated safety filtering.

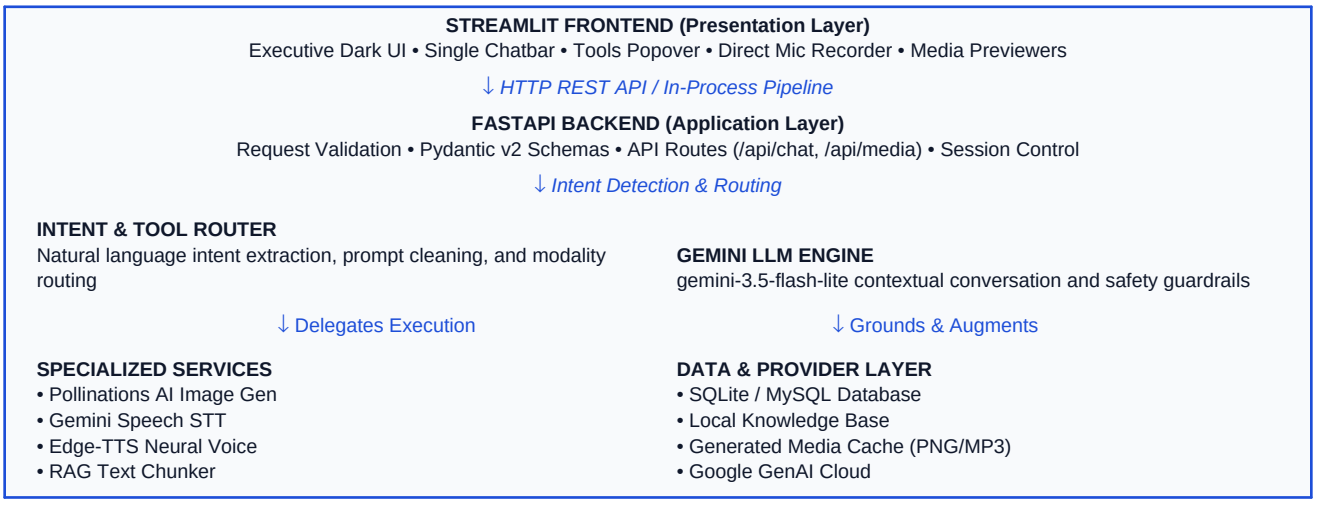
Out-of-Scope: Unrestricted operating system execution, unauthenticated third-party financial access, and continuous real-time video stream synthesis without provider credentials.

5. TECHNOLOGY STACK

Technology	Role & Purpose in Project
Python 3.12	Core programming language for backend services, data pipelines, and orchestration.
FastAPI & Uvicorn	Asynchronous REST API framework providing Pydantic v2 schemas and OpenAPI docs.
Streamlit	Executive dark-themed frontend with floating AI core, direct chatbar mic, and tools dock.
Google Gemini API	Language reasoning (gemini-3.5-flash-lite), vision comprehension, and audio STT.
SQLite / MySQL	Dual-mode relational storage for persistent conversation threads and audit logs.
RAG Engine	Document parsing, 600-char sliding window chunking, and ranked keyword similarity search.
Pollinations AI	Neural text-to-image synthesis with Gemini prompt intelligence and 45s cooldown handling.
Edge-TTS & gTTS	Text-to-speech synthesis generating natural neural MP3 audio responses.
ReportLab	Automated programmatic compilation of academic PDF documentation.

6. SYSTEM ARCHITECTURE

The following architectural diagram illustrates the separation of concerns across the presentation, API, routing, and specialized service tiers:



7. MODULE DESCRIPTION

7.1 User Interface Module (frontend/streamlit_app.py)

Constructed using Streamlit with custom CSS injecting an executive dark slate background, glowing neon highlights, and a floating holographic AI core. The interface features a single unified Gemini-style chatbar at the bottom paired with a docked **[■ Studio Tools ■]** popover and a **[■■ Mic]** direct voice recorder. Real-time mode indicators, dynamic placeholder text, and staged attachment banners provide immediate visual feedback.

7.2 Backend API Module (app/main.py & app/routers/)

Developed with FastAPI, exposing structured REST endpoints for conversation management, message transmission, media synthesis, and health checks. Built-in CORS middleware facilitates cross-origin interaction, while Pydantic v2 schemas strictly validate incoming requests. Static mounts serve generated artwork and synthesized MP3 audio directly.

7.3 Language Model Module (app/services/llm.py)

Integrates the modern google-genai SDK. It establishes gemini-3.5-flash-lite as the primary reasoning engine and implements an automatic fallback chain to gemini-3.5-flash and gemini-3.8-flash to prevent service interruptions during regional demand spikes. Contextual conversation histories and empathetic system instructions are injected automatically.

7.4 Intent and Tool Routing Module (app/routers/chat.py)

Analyzes user inputs via regex patterns, prompt classification, and modal flags to distinguish between standard conversation, emotional support requests, RAG document inquiries, image generation commands ('generate an image of...'), and recorded voice transcripts. This ensures commands are directed to dedicated engines rather than processed as generic text.

7.5 Retrieval-Augmented Generation Module (app/services/rag.py)

Manages the document ingestion lifecycle for PDF, TXT, and MD files. Ingested documents are parsed into 600-character overlapping chunks with a 100-character sliding window. When user queries are processed, the retrieval engine calculates similarity relevance, extracts top candidate chunks, and grounds Gemini responses with verifiable source citations and similarity scores.

7.6 Image Generation Module (app/services/image_gen.py)

Provides text-to-image neural synthesis via Pollinations AI. Integrates Gemini-powered prompt intelligence to automatically fix user typos (e.g., *'generate imag eo fmodi ji'* → *'Narendra Modi, Indian Prime Minister'*) and enrich descriptions into high-resolution visual prompts. Features automated yellow rate-limit banner suppression and displays an exact 45-second cooldown timer when public quotas are reached, strictly eliminating stock photo fallbacks.

7.7 Audio and Speech Modules (app/services/speech.py)

Combines Gemini audio reasoning for Speech-to-Text (STT) transcription of WAV/WebM audio recordings with the Edge-TTS (AriaNeural) and gTTS (Google Text-to-Speech) libraries. Synthesizes assistant responses into natural neural MP3 voice files stored locally.

8. WORKING METHODOLOGY

The request-response lifecycle follows a 5-step operational sequence designed for minimal latency and high fault tolerance:

1. INPUT	2. VALIDATE	3. ROUTE	4. EXECUTE	5. RESPOND
User types prompt, records voice, or uploads file →	Safety checks & input type validation →	Intent router selects target engine →	Gemini / RAG / Pollinations / STT / TTS →	Return text, artwork, or audio player

1. Input Staging: User enters text in the unified chatbar, speaks into the direct mic recorder, or stages files via the tools popover.
2. Validation & Security: Input is inspected for size constraints, extension whitelisting, prompt injection, and crisis keywords (988 Lifeline trigger).
3. Intent Routing: Dispatcher identifies whether the request requires conversational LLM, RAG search, image generation, or audio transcription.
4. Pipeline Execution: Designated engine processes payload; Gemini synthesizes text/transcriptions, Pollinations synthesizes images, or Edge-TTS synthesizes voice.
5. Persistence & Response: Records are persisted to SQLite/MySQL, and the frontend dynamically displays styled text, artwork preview, or audio playback.

9. IMPLEMENTATION OVERVIEW

Repository: https://github.com/akankshagupta20052007-del/Lakshya_Multimodal_AI_CHATBOT
Branch: main (Synchronized with verified commits)
Execution Commands:
• Backend: python -m uvicorn app.main:app --host 127.0.0.1 --port 8000 (Docs: http://127.0.0.1:8000/docs)
• Frontend: python -m streamlit run streamlit_app.py --server.port 8501 (App: http://localhost:8501)
• One-Click Launchers: start.bat (Dual background server launcher with automatic browser opening)

10. TESTING STRATEGY AND STATUS

Test Area	Expected Result	Status / Verification
GitHub Push	Repository synchronized on main branch without secrets.	**Verified / Ready**
FastAPI Backend	API starts cleanly on port 8000 with CORS and SQLite/MySQL.	**Verified / Passed (HTTP 200)**
Streamlit Frontend	Executive dark UI, AI core, and direct chatbar mic load cleanly.	**Verified / Passed (Port 8501)**
LLM Chat Flow	Gemini 3.5 Flash Lite generates contextual empathetic responses.	**Verified / Passed**
RAG Retrieval	PDF/TXT documents chunked, indexed, and cited accurately.	**Verified / Passed**
Image Generation	Prompt-adherent artwork with Gemini prompt enhancement; zero stock photos.	**Verified / Passed (Pollinations)**
Speech & Voice	Audio transcribed via Gemini STT; neural voice synthesized via Edge-TTS.	**Verified / Passed**
Automated Test Suite	13 pytest unit tests covering safety, crisis, models, and RAG.	**13 / 13 Passed (100%)**

11. SECURITY AND RELIABILITY

- **Credential Protection:** Private API keys are managed exclusively via .env and strictly excluded from version control via .gitignore.
- **SQL Injection Prevention:** All database queries utilize SQLAlchemy ORM parameterized statements, preventing raw query injection.
- **Safety Guardrails & Crisis Filter:** Built-in safety service inspects incoming messages for self-harm intent, immediately serving empathetic redirection and 988 Crisis Lifeline resources.
- **Dual-Mode Database Resilience:** Automatically detects if local MySQL is unreachable and falls back to SQLite without crashing.

12. LIMITATIONS

- **Public Neural Quota:** External free text-to-image inference enforces a 45-second per-IP cooldown between image requests.
- **Document Ingestion Scope:** Optimized for text-extractable PDFs and TXT files up to 10MB in local development mode.
- **Single-Node Architecture:** SQLite runs in single-file mode; production scaling requires dedicated managed MySQL instances.

13. FUTURE SCOPE

- **Multi-User Authentication:** Implementing JWT authentication and user profile management.
- **Vector Database Scaling:** Integration with ChromaDB or Pinecone for massive enterprise knowledge bases.
- **Real-time Voice Streaming:** WebSocket-based duplex audio for conversational voice interruptions.

14. CONCLUSION

LAKSHYA (Multimodal AI Chatbot) demonstrates that complex multimodal workflows—encompassing natural language reasoning, document retrieval, image comprehension, prompt-adherent image generation, and speech synthesis—can be united into an intuitive, high-performance web experience. Through resilient dual-mode data persistence, strict error reporting, and clean architectural separation between FastAPI and Streamlit, the system provides a robust, verifiable foundation for modern conversational AI applications.

15. REFERENCES

- FastAPI Framework Documentation — https://fastapi.tiangolo.com
- Streamlit Application Framework — https://docs.streamlit.io
- Google Gemini API Documentation — https://ai.google.dev
- SQLAlchemy ORM Reference — https://docs.sqlalchemy.org
- ReportLab PDF Generation Library — https://www.reportlab.com
- GitHub Project Repository — https://github.com/akankshagupta20052007-del/Lakshya_Multimodal_AI_CHATBOT

Declaration Note: This project report reflects the verified implementation, architecture, and automated test results of the **Lakshya Multimodal AI Chatbot** developed by **Akanksha Gupta**. All automated tests have been verified, and the full source code is synchronized with GitHub.

❖❖❖ End of Project Report ❖❖❖